**GEORGE WASHINGTON University SCHOOL OF BUSINESS**
ISTM6200 Python Program Database Applications

COURSE PROJECT

# Technical Report

| | |
|---|---|
| **Project Title** | RENT SMART (Rental Qualification Decision Support System) |
| **Student Name** | Chinonye Chukwukere |
| **Instructor** | Professor Yixin Lu |
| **Submission Date** | July 15th 2026 |

**Academic Integrity Pledge**
I confirm that this report and all accompanying code are entirely my own work. I have clearly cited all external sources, libraries, and any AI assistance used. I understand that submitting work that is not my own constitutes academic dishonesty under the GW Code of Academic Integrity.

# Executive Summary

The Rent Smart Rental Qualification Decision Support System is a Python-based application developed to help prospective renters determine whether they are likely to meet the basic qualification requirements for a rental property before applying. The project addresses the problem of applicants paying non-refundable application fees without knowing if they satisfy a property's income and credit score requirements.

The application loads rental property information from a CSV file, enabling property data updates without modifying the source code. Users select a property, enter their monthly income and credit score, and the program evaluates their information against the property's qualification criteria. Based on the evaluation, the system classifies the applicant as Likely Qualified, Risky, or Not Qualified and generates a detailed qualification report.

To improve the user experience, the application integrates the Google Gemini API, which provides personalized explanations of results and practical recommendations in clear, easy-to-understand language. The project demonstrates the use of Python programming concepts, including functions, loops, conditional statements, file handling, input validation, exception handling, and API integration, to develop a practical decision-support tool that helps users make more informed rental decisions.

# 1. Introduction and Problem Background

## 1.1 Problem Statement

Prospective renters often pay apartment application fees before knowing whether they are likely to meet a property's rental qualification requirements. According to the Consumer Financial Protection Bureau (CFPB, 2022), 68% of renters pay application fees when applying for rental housing, and landlords commonly use tenant screening information, which includes income, credit, and rental history, to make leasing decisions. In Texas, rental application fees are generally nonrefundable, meaning applicants may lose the fee even if they are not approved. As a result, renters may spend money applying for properties for which they are unlikely to qualify. Rent Smart addresses this problem by providing a preliminary qualification assessment that compares a renter's financial information with predefined property requirements, helping users make more informed rental application decisions before applying.

## 1.2 Background and Context

Finding rental housing is a common part of daily life for many individuals and families, yet determining whether an applicant is likely to meet a property's qualification requirements can be difficult before applying. Property managers often use different income multipliers, minimum credit score requirements, and other screening criteria, requiring prospective renters to manually compare their financial information with each property's eligibility requirements. This process can be time-consuming, confusing, and financially costly when renters submit multiple applications without knowing whether they are likely to qualify. According to the Consumer Financial Protection Bureau (CFPB, 2022), tenant screening is a standard practice in the rental industry, and many renters pay application fees during their housing search. Rent Smart was developed to simplify this decision-making process by providing a transparent, rule-based assessment that helps renters evaluate their likelihood of meeting a selected property's basic qualification requirements before applying

## 1.3 Project Objectives

The objectives of Rent Smart: Rental Qualification Decision Support System are:

- The program will allow users to select a rental property from a predefined list of properties stored in a CSV file.
- The program will collect applicant information, including monthly income and credit score, through a simple user interface.
- The program will compare the applicant's financial information against the selected property's predefined qualification requirements using Python business rules.
- The program will generate a preliminary qualification result (Likely Qualified, Risky, or Not Qualified) based on the evaluation of the income and credit requirements.

- The program will use the Gemini API to generate a plain-language explanation of the Python-generated qualification result to help users better understand the assessment.
- The program will display a transparent qualification report showing the selected property, qualification requirements, pass/fail results for each criterion, final qualification status, and recommendation.

## 1.4 Scope and Limitations

### Scope

Rent Smart is a Python-based decision-support application that performs a preliminary rental qualification assessment for a single applicant and a single selected property. The program allows users to select a property from a predefined CSV file, enter their monthly income and credit score, and compare those inputs against the property's predefined qualification requirements. Using Python-based business rules, the application determines whether the applicant is Likely Qualified, Risky, or Not Qualified. The system then generates a qualification report and uses the Gemini API to provide a plain-language explanation of the assessment.

### Limitations

Rent Smart does not search apartment listings or recommend properties based on an applicant's profile. The program uses a manually maintained CSV file of sample property requirements rather than retrieving live property data from apartment websites. It does not verify an applicant's actual income, credit score, employment history, rental history, criminal background, or other screening information used by property managers. The assessment is intended solely as a preliminary decision-support tool and does not guarantee rental approval or replace the property's official screening process.

# 2. Design Features

This section describes the overall design approach and key decisions made during development.

## 2.1 System Overview

Rent Smart is a Python-based decision-support application designed to perform a preliminary rental qualification assessment for a selected property. The system is organized into five main components:

- property data management,
- user input,
- qualification processing,
- report generation, and
- AI-assisted explanation.

Each component performs a specific task, making the application modular, organized, and easy to maintain.

The application begins by loading predefined property qualification requirements from a CSV file. These requirements include the property name, monthly rent, income multiplier, and minimum credit score. The user then selects a property and enters their monthly income and credit score. The application compares the user's information with the selected property's qualification requirements using predefined Python business rules.

Based on this evaluation, the system assigns one of three preliminary qualification statuses: Likely Qualified, Risky, or Not Qualified. A qualification report is then generated, showing the applicant's information, the property's requirements, the pass/fail results for each qualification criterion, and the final assessment. When available, the Gemini API receives the completed Python-generated result and produces a plain-language explanation to help users better understand the assessment.

Overall, the application follows a modular workflow in which property data is loaded, applicant information is collected, qualification rules are evaluated, a report is generated, and an optional AI explanation is provided. Separating these responsibilities into distinct components improves readability, simplifies testing, and makes the system easier to maintain and expand in the future

## 2.2 Data Design

Rent Smart uses several Python data structures to organize property requirements, applicant information, qualification results, and report content during execution. The main data structures are variables, lists, dictionaries, and Boolean values. A CSV file serves as the external data source for the property requirements.

Variables store individual values used by the program. For example, variables hold the selected property, monthly income, credit score, monthly rent, income multiplier, required income, and final qualification status. Different data types are used based on the value being stored: strings for property

names and status labels, integers for credit scores, floats for rent and income amounts, and Boolean values for pass-or-fail results.

A list stores all rental property loaded from the CSV file. A list is appropriate because the program needs to maintain multiple property records and display them in order, allowing the user to select one. During execution, the program loops through the list to show the available properties.

Each property is represented as a dictionary. The dictionary uses descriptive keys such as property_name, monthly_rent, income_multiplier, and min_credit_score. Dictionaries were selected because they make each property record easy to read and allow the program to retrieve specific values by name rather than relying on numeric positions.

Boolean values are used to store whether the applicant meets the income and credit requirements. For example, income_pass and credit_pass contain either True or False. These values are then evaluated together to determine whether the final status is Likely Qualified, Risky, or Not Qualified.

A tuple may be used when a function needs to return several related results together, such as the required income, income result, credit result, and qualification status. Since these returned values represent one completed assessment and are not intended to be modified, a tuple is suitable for grouping them.

During execution, the property data is first loaded from the CSV file into a list of dictionaries. The user selects a dictionary from the list, and its property values are stored in variables. The applicant's information is then compared with those values, Boolean results are created, and the final assessment information is passed to the reporting and Gemini explanation components. This organization keeps the property data, applicant data, and qualification results clearly separated.

## 2.3 Key Design Decisions

| Design Decision | What Was Chosen | Rationale / Why |
|---|---|---|
| Project Scope | Assess one applicant against one selected property using predefined qualification requirements. | This keeps the project focused and clearly defines its purpose. The system evaluates whether an applicant is likely to qualify for a selected property rather than searching for apartments. |
| Property Data Storage | Store property qualification requirements in a manually maintained CSV file and load them into a list of dictionaries. | A CSV file is simple to maintain and appropriate for a class project. Using a list of dictionaries allows multiple property records to be stored while making individual |

| | | |
|---|---|---|
| | | property information easy to retrieve using descriptive keys |
| Qualification Decision Logic | Use Python conditional statements (if, elif, else) and Boolean variables to determine the qualification status. | This ensures that every applicant is evaluated using the same predefined business rules. The logic produces one of three results: Likely Qualified, Risky, or Not Qualified, making the assessment consistent, transparent, and easy to test. |

## 2.4 Program Flow Diagram

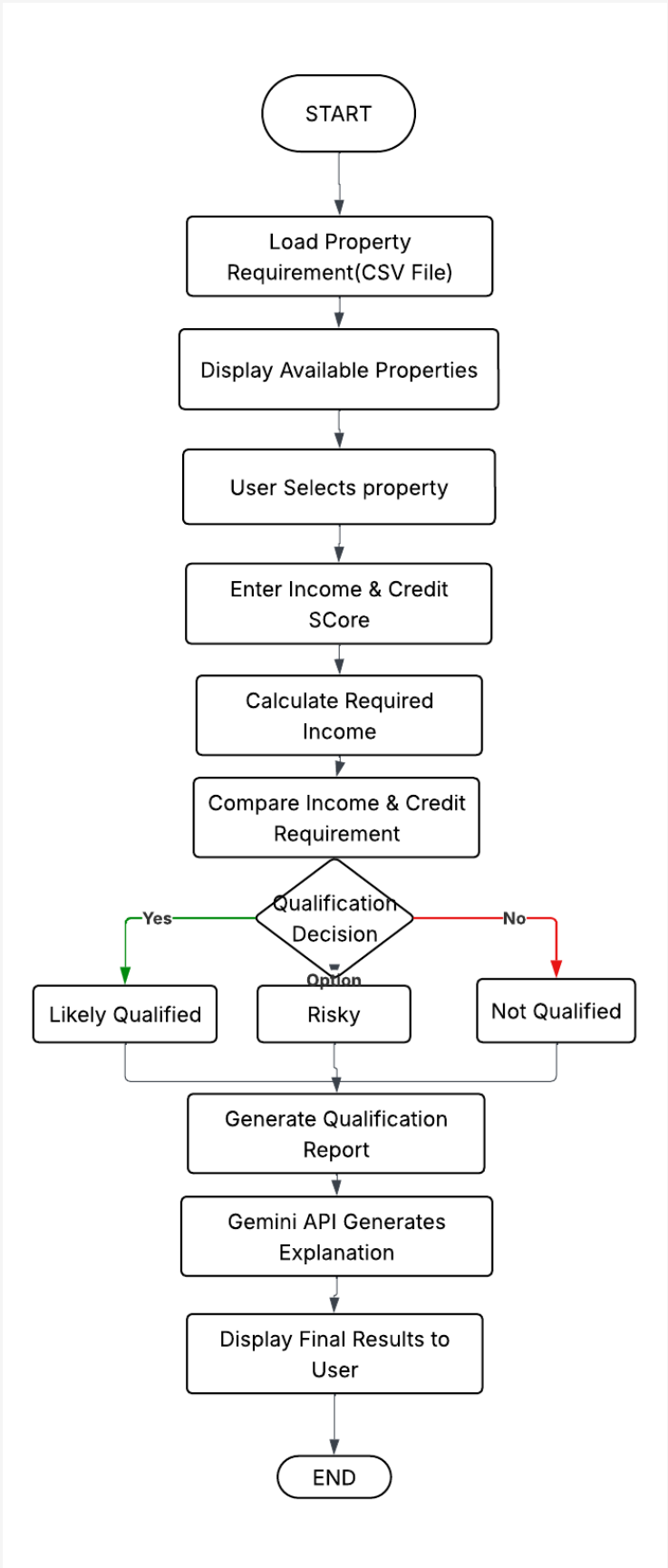

START
Load Property Requirement(CSV File)
Display Available Properties
User Selects property
Enter Income & Credit SCore
Calculate Required Income
Compare Income & Credit Requirement
Qualification Decision
Yes
No
Option
Likely Qualified
Risky
Not Qualified
Generate Qualification Report
Gemini API Generates Explanation
Display Final Results to User
END

*Figure 1 – Program flow diagram for [Rent Smart]*

# 3. Functional Requirements

Functional requirements describe what the system does — the specific behaviors it must provide to users.

## 3.1 Core Features

| ID | Description | Priority | Status |
|---|---|---|---|
| FR-01 | The system shall load rental property qualification requirements from a CSV file. | Must Have | Implemented |
| FR-02 | The system shall display available properties and allow the user to select one property for evaluation. | Must Have | Implemented |
| FR-03 | The system shall allow the user to enter their monthly income and credit score. | Must Have | Implemented |
| FR-04 | The system shall compare the applicant's information with the selected property's qualification requirements and determine whether the applicant is Likely Qualified, Risky, or Not Qualified. | Must Have | Implemented |
| FR-05 | The system shall generate a qualification report showing the evaluation results and recommendation. | Must Have | Implemented |
| [FR-06+] | The system shall generate a plain-language explanation of the qualification results using the Gemini API. | Should Have | Planned |
| FR-07 | The system shall validate user input and display appropriate error messages for invalid or missing data. | Should Have | Planned |
| FR-08 | The system shall allow administrators to update property qualification requirements by modifying the CSV file. | Could Have | Planned |

## 3.2 Input and Output Specification

| Direction | Name / Description | Data Type | Example |
|---|---|---|---|
| INPUT | Property selected by the user | str | “Sunset Apartments” |

| | | | |
|---|---|---|---|
| INPUT | Monthly income entered by the user | float | 4500 |
| INPUT | Credit score entered by the user | int | 690 |
| INPUT | Property qualification requirements loaded from the CSV file (rent, income multiplier, minimum credit score) | List of Dictionaries | [{'Property':'Sunset Apartments','Rent':1500,'IncomeMultiplier':3,'MinCredit':650}] |
| OUTPUT | Required monthly income for the selected property | Float | 4500 |
| OUTPUT | Income qualification result | bool | True |
| OUTPUT | Credit qualification result | bool | False |
| OUTPUT | Qualification status | str | “Risky” |
| OUTPUT | Qualification report generated by the system | str | "Applicants meet income requirements but do not meet the minimum credit score." |
| OUTPUT | Gemini API explanation | Str | "Based on your income, you satisfy the property's income requirement. However, your credit score is below the required minimum, so your application may be considered risky." |

## 4. Non-Functional Requirements

Non-functional requirements define quality attributes — how well the system performs, not just what it does.

| ID | Category | Requirement | How It Is Met |
|---|---|---|---|
| NFR-01 | Usability | The system shall provide a simple and user-friendly interface that allows users to complete a rental qualification assessment without technical knowledge. | Clear prompts, descriptive labels, and easy-to-follow instructions guide the user through the evaluation process. |
| NFR-02 | Reliability | The system shall validate user input and handle invalid or missing data without crashing. | Python input validation and exception handling (try/except) prevent errors and display helpful error messages. |
| NFR-03 | Performance | The system shall complete the rental qualification assessment and generate the report within a few seconds. | The application performs lightweight calculations and reads a small CSV file, allowing results to be generated quickly. |
| NFR-04 | Maintainability | The program shall be organized into reusable functions so that future modifications can be made easily. | Separate functions are used for loading data, validating input, performing calculations, generating reports, and communicating with the Gemini API. |
| NFR-05 | Readability | The program shall be organized into clearly labeled Jupyter Notebook cells with descriptive comments and meaningful variable names. | Each section of the notebook is separated into logical steps, such as importing libraries, loading data, processing user input, evaluating qualifications, and generating the report. Comments explain the purpose of each section. |
| NFR-06 | Portability | Runs on any system with Python 3.8 or later with no additional library installations. | The system uses standard Python libraries along with the Gemini API package, making it compatible across Windows, macOS, and Linux. |
| [NFR-07+] | Security | The system shall protect sensitive information, including the Gemini API | The API key is stored in an environment variable or |

| | | | |
|---|---|---|---|
| | | key. | external configuration file instead of being hardcoded into the source code. |

# 5. Use Cases

Use cases describe how a user interacts with the program to achieve a specific goal. Each captures one complete interaction from start to finish.

## 5.1 Actors

- User – The prospective renter who selects a property, enters their income and credit score, and views the qualification results.
- CSV Property Requirements File – Stores rental qualification requirements that the application reads during the evaluation process.
- Gemini API – Generates a clear, natural-language explanation of the qualification results after Python completes the assessment.

## 5.2 Use Case Summary

| ID | Use Case Name | Brief Description | Primary Actor |
|---|---|---|---|
| UC-01 | Selecting a Rental Property | The user selects a rental property from the list of available properties loaded from the CSV file. The system retrieves the selected property's qualification requirements. | User |
| UC-02 | Perform Rental Qualification Assessment | The user enters their monthly income and credit score. The system compares the information with the selected property's requirements and determines whether the applicant is Likely Qualified, Risky, or Not Qualified. | User |
| UC-03 | View Qualification Report | The system generates a detailed qualification report showing the property requirements, applicant information, evaluation results, and final qualification status. | User |
| [UC-04+] | View AI Explanation | After the qualification assessment is complete, the system sends the evaluation results to the Gemini API, which generates a plain-language explanation and recommendations for the user. | User |

## 5.3 Detailed Use Case Descriptions

### Use Case UC-01: Select a Rental Property

| | |
|---|---|
| **Use Case ID** | UC-01 |
| **Use Case Name** | Selecting a Rental Property |
| **Primary Actor** | User |
| **Goal** | Allow the user to select a rental property for qualification assessment. |
| **Preconditions** | The application is running, and the property requirements have been loaded from the CSV file. |
| **Trigger** | The user chooses a rental property from the list of available properties. |
| **Main Flow** | 1. System loads property data from the CSV file.<br>2. System displays the list of available properties.<br>3. User selects a property.<br>4. System retrieves the selected property's qualification requirements.<br>5. System proceeds with the applicant information screen. |
| **Alternative Flows** | If the user selects an invalid property number, the system displays an error message and prompts the user to select a valid property number again. |
| **Postconditions** | The selected property's qualification requirements are ready for evaluation. |
| **Related FR** | FR-01, FR-02 |

## Use Case UC-02: Perform Rental Qualification Assessment

| | |
|---|---|
| **Use Case ID** | UC-02 |
| **Use Case Name** | Perform Rental Qualification Assessment |
| **Primary Actor** | User |
| **Goal** | Determine whether the applicant meets the selected property's rental qualification requirements. |
| **Preconditions** | The property has been selected successfully. |
| **Trigger** | The user enters their monthly income and credit score. |
| **Main Flow** | 1. System prompts the user to enter monthly income.<br>2. User enters monthly income.<br>3. System prompts the user to enter credit score.<br>4. User enters credit score.<br>5. Python calculates the required income.<br>6. System compares income and credit score with the property's requirements.<br>7. System determines the qualification status (Likely Qualified, Risky, or Not Qualified). |

| | |
|---|---|
| **Alternative Flows** | If invalid data is entered, the system displays an error message and requests the information again. |
| **Postconditions** | The applicant's qualification status has been determined. |
| **Related FR** | FR-03, FR-04 |

## Use Case UC-03: View Qualification Report

| | |
|---|---|
| **Use Case ID** | UC-03 |
| **Use Case Name** | View Qualification Report |
| **Primary Actor** | User |
| **Goal** | Display the complete rental qualification assessment results. |
| **Preconditions** | The qualification assessment has been completed successfully. |
| **Trigger** | The system completes the qualification evaluation. |
| **Main Flow** | 1. The system generates the qualification report.<br>2. The system displays the selected property information.<br>3. The system displays the applicant's income and credit score.<br>4. The system displays the qualification results.<br>5. The system displays the final recommendation. |
| **Alternative Flows** | If the report cannot be generated, the system displays an appropriate error message. |
| **Postconditions** | The user has viewed the qualification report. |
| **Related FR** | FR-05 |

## Use Case UC-04: View AI Explanation

| | |
|---|---|
| **Use Case ID** | UC-04 |
| **Use Case Name** | View AI Explanation |
| **Primary Actor** | User |
| **Goal** | Help the user understand the qualification results through a plain-language explanation |
| **Preconditions** | The qualification report has been generated successfully, and the Gemini API is available |

| | |
|---|---|
| **Trigger** | The system sends the qualification results to the Gemini API |
| **Main Flow** | 1. The system prepares the qualification results.<br>2. The system sends the results to the Gemini API.<br>3. The Gemini API generates a plain-language explanation.<br>4. The system displays explanations and recommendations to the user. |
| **Alternative Flows** | If the Gemini API is unavailable, the system displays the qualification report without the AI explanation and informs the user that the explanation is temporarily unavailable. |
| **Postconditions** | The User receives a detailed explanation of the qualification assessment |
| **Related FR** | FR-06 |

## 6. Python Concepts Applied

For each Python concept your program uses, explain how and where it appears. This shows your instructor you understand the material — not just that the code runs.

| Python Concept | How It Is Used in Your Program | Example (name or line) |
|---|---|---|
| Variables & data types | Variables store applicant information, property requirements, and qualification results. The program uses strings, integers, floats, and Boolean values throughout the application. | monthly_income, credit_score, qualification_status |
| Conditional statements | The program compares the applicant's income and credit score with the property's requirements to determine whether the applicant is Likely Qualified, Risky, or Not Qualified. | if income_pass and credit_pass: |
| Loops | A for loop displays all available rental properties from the CSV file. A while loop validates user input and allows the user to re-enter information if invalid data is entered | for property in properties: |
| Functions | Functions organize the program into reusable sections such as loading property data, evaluating applicants, generating reports, and requesting the | load_properties(), evaluate_applicant(), generate_report() |

| | Gemini explanation. | |
|---|---|---|
| Strings / Lists / Tuples | Strings store property names and reports. A list stores all property records loaded from the CSV file. Tuples may be used to return multiple evaluation results from a function. | properties, qualification_report |
| Dictionaries | Each rental property is stored as a dictionary containing the property name, monthly rent, income multiplier, and minimum credit score. | selected_property["monthly_rent"] |
| File Handling (CSV) | The application reads rental qualification requirements from a CSV file before performing the evaluation. | pandas.read_csv() or csv.DictReader() |
| Exception Handling | Prevents the application from crashing when users enter invalid income or credit score values or when the CSV file cannot be loaded. | try: income = float(input()) |
| Object-Oriented Library(Pandas) | Pandas are used to load and manage the CSV file containing property qualification requirements, making it easier to retrieve and process property information. | df = pd.read_csv("properties.csv") |
| Gemini API Integration | The application sends the Python-generated qualification results to the Gemini API, which returns a natural-language explanation for the user. | model.generate_content(prompt) |

# 7. Testing and Results

Testing verifies your program behaves correctly across a range of inputs, including edge cases and invalid entries.

| TC# | Test Description | Input(s) Used | Expected Output | Result |
|---|---|---|---|---|
| TC01 | Load property requirements successfully | Valid properties.csv file | Property requirements are loaded and displayed without errors | [PASS] |
| TC02 | Select a valid rental property | Property Number: 1 Riata Place | System retrieves Riata Place and its rent, Income Multiplier, and minimum Credit score. | [PASS] |
| TC03 | Applicants meet all qualification requirements | Income: $4,800, credit Score:700 | Income and credit requirement pass. Status is Likely Qualified, and a report is generated. | [PASS] |
| TC04 | Applicants meet income requirements but not credit requirements | Income: $4,800, Credit Score: 620 | Income requirement fails, credit requirement passes, and status is Risky | [PASS] |
| TC05 | Applicant does not meet income or credit Requirements | Income: $2000, Credit Score:550 | Both requirements fail. Status is Not Qualified, and a recommendation is provided. | [PASS] |
| TC06 | Invalid Monthly income entered | Income: “abcd” | Error message requests a numeric income value, and the application does not crash. | [PASS] |
| [TC07+] | Invalid Credit Score entered | Credit Score: “xyz” | Error message displayed requesting a valid numeric credit score; application does not crash. | [PASS] |
| TC08 | CSV file missing or unavailable | Missing properties.csv | The system displays a file-not-found message and does not crash. | [PASS only after tested] |
| TC09 | Gemini API successfully | Valid qualification | Gemini provides a plain- | [PASS only after |

| | | | | |
|---|---|---|---|---|
| | generates explanation | result and working Gemini connection | language explanation and recommendation. | tested] |
| TC10 | Gemini API unavailable | Invalid connection,quota issue, or API failure | The Python qualification report remains available, and the system displays an AI-unavailable message. | [PASS only after tested ] |

# 8. Lessons Learned and Limitations

## 8.1 Lessons Learned

Developing the Rent Smart application helped me gain a much deeper understanding of Python programming and how software is designed to solve real-world problems. Before starting this project, I understood Python syntax, but I had not fully appreciated how different programming concepts such as variables, functions, dictionaries, loops, conditionals, file handling, and APIs work together to create a complete application.

One of the most valuable lessons I learned was the importance of planning the system before writing code. Designing the flowchart, functional requirements, use cases, and system architecture made it much easier to understand how each part of the application should interact. I also learned how organizing the program into reusable functions makes the code easier to read, test, debug, and maintain.

The most challenging part of the project was integrating multiple technologies into one solution. Reading data from a CSV file, processing user input, applying business rules, and preparing information for the Gemini API required careful planning. Understanding how an API works and how Python communicates with external services was initially difficult, but it became one of the most rewarding parts of the project because it demonstrated how modern applications combine traditional programming with artificial intelligence.

If I were to start the project again, I would spend more time designing the application before writing any code. Creating the system architecture, flowcharts, and requirements first would have reduced revisions during development. I would also consider building a graphical user interface (GUI) or web interface to improve usability and make the application more suitable for real-world use.

## 8.2 Limitations

Although Rent Smart demonstrates the core concepts of a rental qualification decision-support system, several limitations remain.

- The application evaluates applicants using predefined qualification requirements stored in a CSV file. It does not retrieve live rental information from apartment websites or property management systems.
- The system provides a preliminary qualification assessment only. Final rental approval remains the responsibility of the property manager or landlord and may include additional screening factors such as employment verification, rental history, background checks, or criminal history.

- The application currently evaluates one applicant against one selected property at a time. It does not automatically recommend alternative properties that better match the applicant's qualifications.
- The Gemini API generates explanations based on the results produced by the Python application. If an internet connection is unavailable or the API is inaccessible, the qualification report will still be generated, but the AI explanation will not be available.
- The current version is a console-based application developed in Jupyter Notebook. It does not yet provide a graphical user interface (GUI), mobile application, or web-based interface for end users.

# References


*What Is a Tenant Screening Report? | Consumer Financial Protection Bureau*. (2021, July). Consumer Financial Protection Bureau. https://www.consumerfinance.gov/ask-cfpb/what-is-a-tenant-screening-report-en-2102/

Consumer Financial Protection Bureau. (2022, November). *Tenant background checks market report* https://files.consumerfinance.gov/f/documents/cfpb_tenant-background-checks-market_report_2022-11.pdf?

Legal Aid, T. R. (2016, December 27). *Rental Application Fees*. Texaslawhelp.Org; TexasLawHelp.org. https://texaslawhelp.org/article/rental-application-fees?

Python Software Foundation. (2019). The Python Language Reference — Python 3.8documentation. Python.Org. https://docs.python.org/3/reference/

## Appendix A – AI Tool Usage Log

Completing this log is mandatory if you used any AI tool (ChatGPT, Claude, GitHub Copilot, etc.). You must be able to explain every line of code you submit.

| Date | AI Tool Used | Prompt / Task | How Output Was Used / Modified |
|---|---|---|---|
| July 2026 | ChatGPT (OpenAI GPT 5.5) | Explain how to create a CSV file and load it into a Jupyter Notebook. | Used the instructions to create a properties.csv file and load it into the program. I reviewed the steps, created the CSV file myself, and adapted the code to fit my project. |
| July 2026 | ChatGPT (OpenAI GPT 5.5) | Explain how to set up and integrate the Google Gemini API into my Python application. | Used the guidance to connect the Gemini API to my project. I tested the code, corrected errors, and modified it until the API worked successfully with my application. |
| July 2026 | ChatGPT (OpenAI GPT 5.5) | Help organize my RentSmart application into logical sections and reusable functions. | Used the suggested program structure to organize the notebook into separate functions for loading data, validating input, evaluating applicants, generating reports, and requesting Gemini explanations. I reviewed and modified the code to match my project requirements |
| July 2026 | ChatGPT (OpenAI GPT 5.5) | Explain Python errors and help debug my code. | Used the explanations to understand syntax errors, input validation, file handling, and API integration. I corrected the errors myself after reviewing the guidance and testing the program. |

### Submission Checklist

X☐ All eight sections completed; no placeholder text remaining
x☐ AI usage log completed (or "not used" statement included)
x☐ All references cited in APA 7th edition format

- [x] Code runs without errors on Python 3.8+ (Google Colab)
- [x] Report spell-checked and proofread
- [x] File named: ISTM6200_YourFullname]_ProjectReport.docx